

PROGRAMMING FOR DATA ANALYTICS 2

PDAN8412/w

PART 1: RECURRENT NEURAL NETWORKS

Author Identification Using a Long Short-Term Memory Network

Dataset: TextAge Historical Text Corpus

Source: Kaggle

Student Number: ST10123828

Student Name: Andiswa Sibusisiwe Phewa

Part 1 Report

Executive summary

This report presents the complete Part 1 analysis for a large book publisher that wants an interactive model capable of guessing an author from a passage of writing. The problem is treated as closed-set multiclass authorship attribution. The TextAge historical text corpus was selected because it contains text passages together with author and title metadata and is large enough to support a meaningful recurrent neural network experiment. The downloaded large file contained 91,918 records, 667 author labels, 914 titles and 15 document categories, with no missing values in the seven supplied columns.

Apache Spark was used to load, inspect and clean the data. Seven frequent named authors were retained, producing 13,624 source records and 13,149 records after removing eight very short fragments and 467 exact duplicate author-text records. A title-aware split created 9,294 training records, 1,869 validation records and 1,986 test records, with zero author-title overlap between the three partitions. This was important because random passage-level splitting could allow text from the same literary work to appear in both training and evaluation data.

Two neural models were trained in PyTorch. The baseline LSTM obtained a validation macro F1 score of 0.3704. The improved bidirectional LSTM used stronger dropout, a larger embedding, class-weighted loss and a lower learning rate. Although its validation accuracy was slightly lower, its validation macro F1 increased to 0.4495, so it was selected according to the planned balanced model-selection criterion. On the untouched test set, the final model achieved 43.91% accuracy, 0.4429 macro F1 and 47.79% balanced accuracy. William Caxton was the strongest class with an F1 score of 0.9040, while Daniel Defoe was the weakest at 0.1931. The results support the model as an educational prototype author-guessing bot, but not as a production or forensic authorship-attribution system.

Table of Contents

Executive summary	2
Table of Figures.....	4
Table of Tables.....	4
1. Introduction and client problem	5
1.1 Problem context.....	5
1.2 Why an LSTM is appropriate	5
1.3 Objectives	5
2. Dataset evaluation and justification.....	6
2.1 Dataset source and structure	6
2.2 Suitability for authorship classification.....	7
2.3 Data-quality risks and common pitfalls.....	7
3. Analysis plan	8
4. Exploratory Data Analysis and cleaning.....	8
4.1 Full-dataset distributions.....	8
4.2 Author subset and cleaning results	11
4.3 Title diversity and leakage risk	13
4.4 Text-length analysis.....	14
4.5 EDA conclusion.....	15
5. Feature selection and data preparation	15
5.1 Feature and target selection.....	15
5.2 Tokenisation and vocabulary	15
5.3 Work-aware split and leakage checks.....	16
6. Model training and improvement.....	17
6.1 Baseline LSTM.....	17
6.2 Improved bidirectional LSTM.....	19
6.3 Validation-led model selection.....	21
7. Final model evaluation and interpretation	22
7.1 Final test metrics.....	22
7.2 Per-author performance.....	22
7.3 Confusion-matrix analysis.....	23
7.4 Final author-guessing demonstration	24
8. Discussion	24
8.1 Did the improvement work?	24

8.2 Relationship between EDA and final performance.....	24
8.3 Limitations.....	25
8.4 Possible improvements.....	25
9. Conclusion and client recommendation	25
10. Reproducibility and code attribution	26
References	27

Table of Figures

Figure 1 Notebook evidence confirming the local environment, Spark execution and the downloaded dataset profile	7
Figure 2 Fifteen most frequent author labels in the complete TextAge dataset.	9
Figure 3 Document category distribution in the complete TextAge dataset.	10
Figure 4 Notebook output showing the complete cleaning sequence and the final author-level record and title counts.....	11
Figure 5 Notebook output showing the complete cleaning sequence and the final author-level record and title counts.....	11
Figure 6 Final cleaned modelling records for the seven selected authors.	12
Figure 7 Distinct literary works available for each selected author.....	13
Figure 8 Character-length distribution after cleaning.	14
Figure 9 Word-count distribution after cleaning.	14
Figure 10 Record counts in the work-aware training, validation and test partitions.	16
Figure 11 Notebook evidence showing the split sizes, zero author-title overlap, vocabulary size, sequence	16
Figure 12 Notebook evidence for the baseline LSTM architecture and best validation result.	17
Figure 13 Baseline LSTM training and validation accuracy across epochs.	18
Figure 14 Baseline LSTM training and validation loss across epochs.	18
Figure 15 Notebook evidence for the improved bidirectional LSTM and its validation result.	19
Figure 16 Improved bidirectional LSTM training and validation accuracy across epochs.....	20
Figure 17 Improved bidirectional LSTM training and validation loss across epochs.....	20
Figure 18 Validation comparison of the baseline and improved models.	21
Figure 19 Notebook output summarising the final untouched test-set performance.	22
Figure 20 Normalised confusion matrix for the final test set.....	23
Figure 21 Held-out demonstration of the final author-prediction function.....	24

Table of Tables

Table 1 Dataset profile and quality summary.....	6
Table 2 Analysis plan for Part 1	8
Table 3 Model configuration comparisons	17
Table 4 Per-author performance on the untouched test set.	22

1. Introduction and client problem

1.1 Problem context

The client is a large book publisher that wants a model for fans to interact with by submitting a passage and receiving a predicted author. The task is therefore an authorship-attribution problem in which the model must learn patterns associated with each candidate writer. Computational authorship attribution has traditionally relied on measurable linguistic characteristics and machine-learning classification, but the reliability of a result depends strongly on how text is represented, how candidate authors are defined and how evaluation data are separated from training data (Koppel, Schler and Argamon, 2009; Stamatatos, 2009).

For this portfolio the problem is deliberately closed-set. The model is not expected to identify any author in the world. It must choose among seven authors represented in the training data. This is appropriate for a prototype entertainment application, but the closed candidate set is also an important limitation when interpreting the final score.

1.2 Why an LSTM is appropriate

Written language is sequential: the interpretation of a word depends partly on the words that occur before and around it. Long Short-Term Memory networks were designed to improve recurrent learning over longer sequences by using input, forget and output gates to control the flow of information through a cell state (Hochreiter and Schmidhuber, 1997). The PyTorch LSTM implementation used in this project follows this gated recurrent structure and supports both unidirectional and bidirectional processing (PyTorch, 2026a).

An LSTM is therefore suitable for the client problem because it can learn ordered patterns from sequences of tokens rather than treating a passage as an unordered bag of words. However, the model can still learn topic, historical period or book-specific vocabulary in addition to personal style, so the analysis has to control obvious sources of leakage and discuss these confounding factors honestly.

1.3 Objectives

1. Evaluate the quality and suitability of the TextAge dataset for LSTM-based author classification.
2. Plan the analysis before modelling, including EDA, feature preparation, model training, evaluation and reporting.
3. Use Spark to inspect and clean the large dataset and conduct exploratory analysis.
4. Create a defensible author subset containing more than 10,000 usable records.
5. Prevent author-title groups from appearing in more than one data partition.
6. Train a baseline LSTM and a justified improved model.
7. Compare models using validation data and balanced metrics before touching the test set.
8. Evaluate the selected model on the untouched test set and demonstrate the final author-guessing function.

2. Dataset evaluation and justification

2.1 Dataset source and structure

The selected dataset is TextAge: A Historical Text Corpus Dataset, hosted on Kaggle. The data card describes a historical corpus spanning 1473 to 1928 and containing text, document category, publication date, title, author, decade and text length fields (Hiray et al., 2025). The downloaded `textage\_large.csv` file was inspected directly rather than assuming that the online description and local file were identical.

Table 1 Dataset profile and quality summary.

Quality measure	Observed result	Relevance
Downloaded records	91,918	Exceeds the 10,000-record assessment minimum
Columns	7	Text plus document and author metadata
Unique author labels	667	Large candidate pool before filtering
Unique titles	914	Enables work-aware splitting
Document categories	15	Books dominate the full corpus
Date range	1473 to 1928	Strong historical variation must be considered
Missing or blank values	0	No required-field loss at the initial quality check
Final cleaned modelling records	13,149	Remains above the assessment minimum

```
Notebook output: environment, Spark execution and dataset profile
Python version: 3.13.9
Java: openjdk version "17.0.15"
PyTorch version: 2.14.0+cpu
Spark version: 4.2.0

Spark test output:
+---+
| id|
+---+
| 0|
| 1|
| 2|
| 3|
| 4|
+---+

Dataset rows: 91,918
Columns: 7
Unique authors: 667
Unique titles: 914
Unique categories: 15
Date range: 1473 to 1928
Missing/blank values in all seven columns: 0
```

Figure 1 Notebook evidence confirming the local environment, Spark execution and the downloaded dataset profile

Figure 1 confirms that the analysis actually ran with Java 17, Spark 4.2.0 and PyTorch on the local machine. It also confirms the central source-level properties used throughout this report. The Kaggle data card reports 91,919 records, whereas the downloaded large CSV contains 91,918 records. This one-record difference is minor, but documenting it avoids presenting online metadata as if it were a direct observation from the file.

2.2 Suitability for authorship classification

The two essential fields are `text` and `author`. The text provides the ordered input sequence and the author provides the class label. The `title` field is not used as a predictive feature because it could directly reveal book-specific information, but it is valuable for building a safer evaluation split. Similarly, category, date and decade are retained for analysis but excluded from the neural-network input so that the model cannot use those metadata fields as shortcuts.

The dataset is also large enough to support a seven-author experiment after cleaning. This is important because a very small author subset could produce unstable model estimates, while attempting all 667 authors would create an unnecessarily large classification task for a local CPU-based portfolio experiment.

2.3 Data-quality risks and common pitfalls

A dataset can have no missing values and still contain modelling risks. Exact duplicate passages can inflate performance if copies are distributed across training and testing. Very short fragments may contain too little stylistic evidence. Unequal class sizes can make overall accuracy appear stronger than the model's performance on smaller classes. Most importantly, passages from the same book may share character names, locations and topic vocabulary, which can make a random row split over-optimistic.

Authorship-attribution research repeatedly emphasises that evaluation design and the amount and type of available text are central to valid conclusions (Stamatatos, 2009; Koppel, Schler and Argamon, 2009).

Historical variation is another concern. William Caxton and William Shakespeare are separated by centuries from writers such as Charles Dickens and Rudyard Kipling. A neural model may therefore learn historical vocabulary and orthography as well as individual style. This does not make the dataset unsuitable, but it means that a high classification score must not automatically be interpreted as pure stylistic recognition.

3. Analysis plan

The analysis was planned before the final model was selected. The purpose of the plan was to make each modelling decision traceable to either the assessment requirement, an observed data property or a clear evaluation principle. The plan was intentionally kept focused because the assessment requires a comprehensive but succinct planning section.

Table 2 Analysis plan for Part 1

Planned stage	Planned approach
Exploratory data analysis	Use Spark to confirm schema, dimensions, missing values, author frequencies, category distribution, duplicates, text lengths and title diversity. Link each finding to a later modelling decision.
Feature selection and preparation	Use text as the predictive feature and author as the target. Exclude metadata from model inputs. Lowercase and tokenize the text, keep punctuation tokens, build vocabulary from training data only and pad/truncate to a fixed sequence length.
Data splitting	Assign complete author-title groups to train, validation and test partitions. Require all seven authors in every partition and verify zero group overlap.
Baseline training	Train a simple Embedding + LSTM classifier using cross-entropy loss and Adam. Monitor validation loss and macro F1.
Improvement	Introduce a larger embedding, bidirectional LSTM, stronger dropout, class-weighted loss and lower learning rate. Compare against the baseline.
Evaluation	Select the model using validation macro F1. Evaluate the selected model once on the untouched test set using accuracy, macro precision, macro recall, macro F1, balanced accuracy, per-author scores and a normalised confusion matrix.
Reporting	Place all key notebook outputs and figures in the report, explain their meaning, state limitations and demonstrate the final prediction function.

4. Exploratory Data Analysis and cleaning

4.1 Full-dataset distributions

The first EDA step examined the complete source rather than immediately filtering to the final authors. This provided context for author frequency and document-type composition and confirmed that the dataset offered enough candidate writers to construct a meaningful subset.

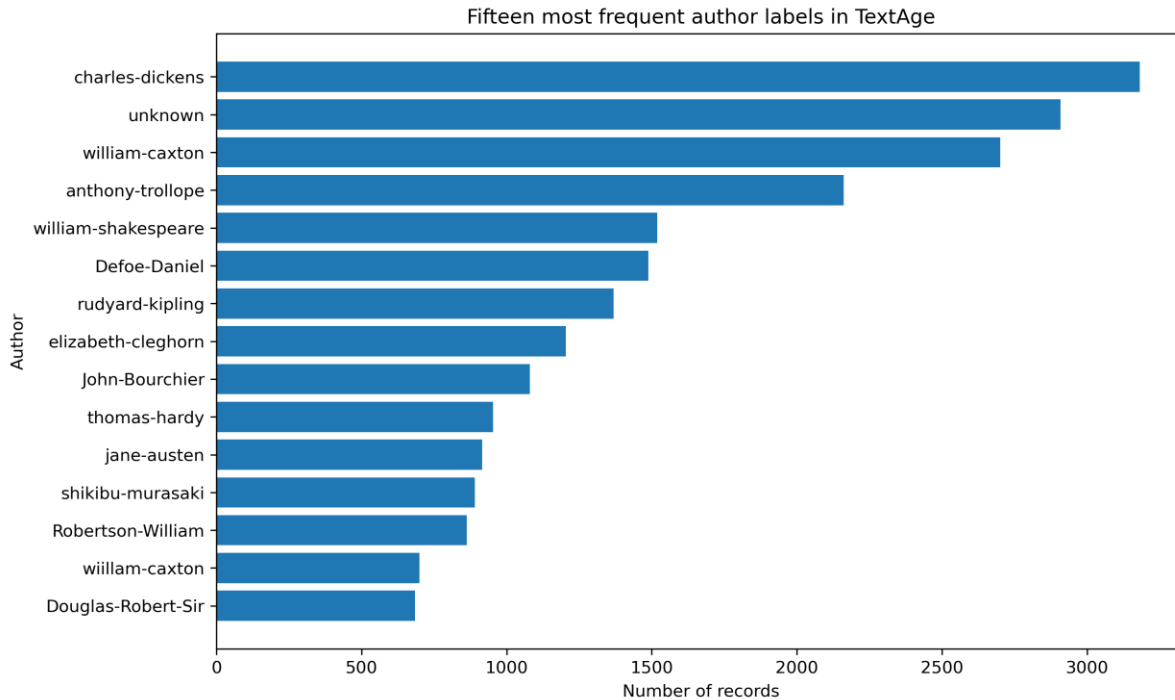


Figure 2 Fifteen most frequent author labels in the complete TextAge dataset.

Charles Dickens is the largest named author class with 3,182 records, followed by William Caxton with 2,701 and Anthony Trollope with 2,161. The `unknown` label is also frequent, but it was excluded because it cannot act as a meaningful named-author target. The EDA also revealed variant labels such as `william-caxton`, which demonstrates that author-label consistency is a real dataset consideration even though the seven selected labels were kept as exact, verified values.

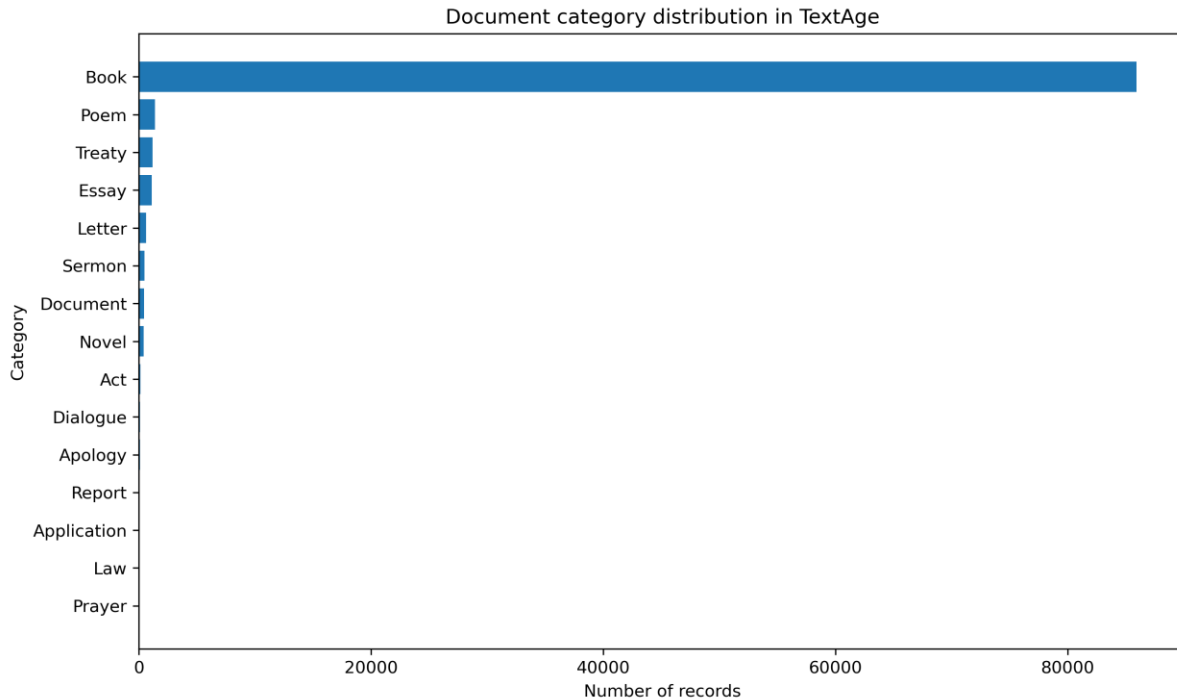


Figure 3 Document category distribution in the complete TextAge dataset.

The full corpus is dominated by the Book category, with much smaller counts for poems, treaties, essays, letters and other document types. This reduces genre diversity in the dominant part of the dataset and means that category imbalance should be considered when generalising the findings beyond book-length historical text.

4.2 Author subset and cleaning results

Seven frequent named authors were selected: Charles Dickens, William Caxton, Anthony Trollope, William Shakespeare, Daniel Defoe, Rudyard Kipling and Elizabeth Cleghorn Gaskell. Together they contributed 13,624 source records before cleaning.

Notebook output: cleaning and final author quality			
Cleaning summary			
Selected source records	13,624		
After required-field checks	13,624		
After short-text filter	13,616		
After exact duplicate removal	13,149		
Final modelling records	13,149		
Ambiguous cross-author text groups	0		
Final modelling records by author			
Charles Dickens	3,181	titles: 7	
William Caxton	2,232	titles: 17	
Anthony Trollope	2,161	titles: 7	
William Shakespeare	1,519	titles: 8	
Daniel Defoe	1,488	titles: 6	
Rudyard Kipling	1,366	titles: 14	
Elizabeth Cleghorn Gaskell	1,202	titles: 14	

Figure 4 Notebook output showing the complete cleaning sequence and the final author-level record and title counts.

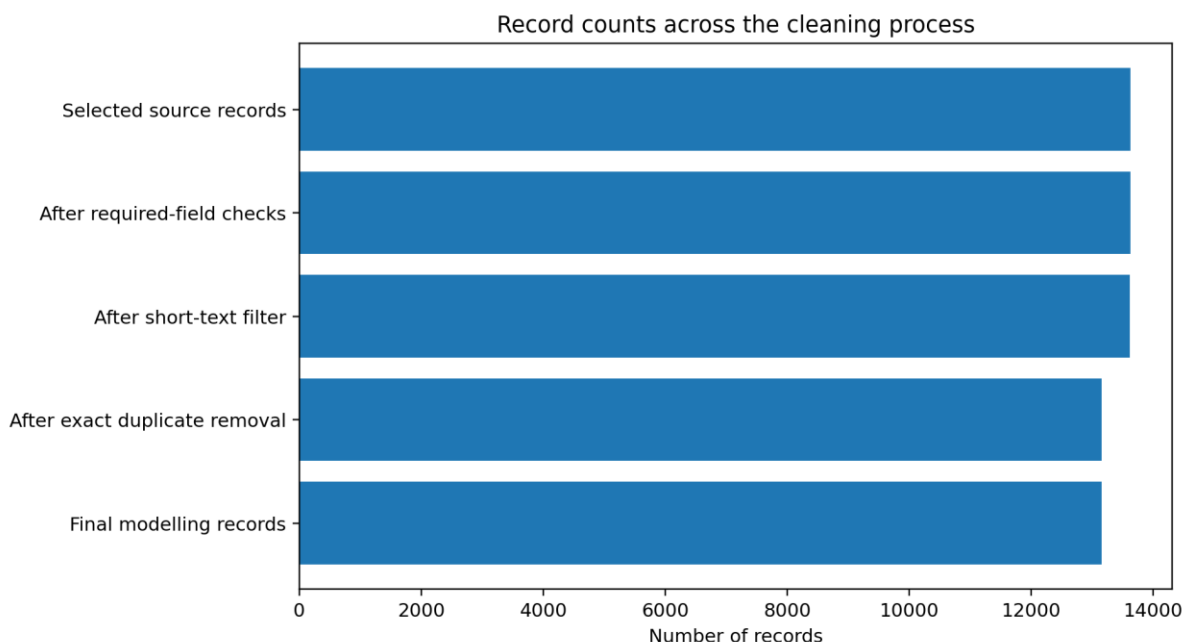


Figure 5 Notebook output showing the complete cleaning sequence and the final author-level record and title counts.

All 13,624 selected records had the required text, author and title fields. Eight passages were removed because they contained fewer than 200 characters. The larger reduction occurred when exact duplicate

author-text pairs were removed: the count fell from 13,616 to 13,149, a reduction of 467 rows. No identical passage was associated with more than one selected author. The final dataset therefore remained above the required 10,000 records while reducing one important source of evaluation inflation.

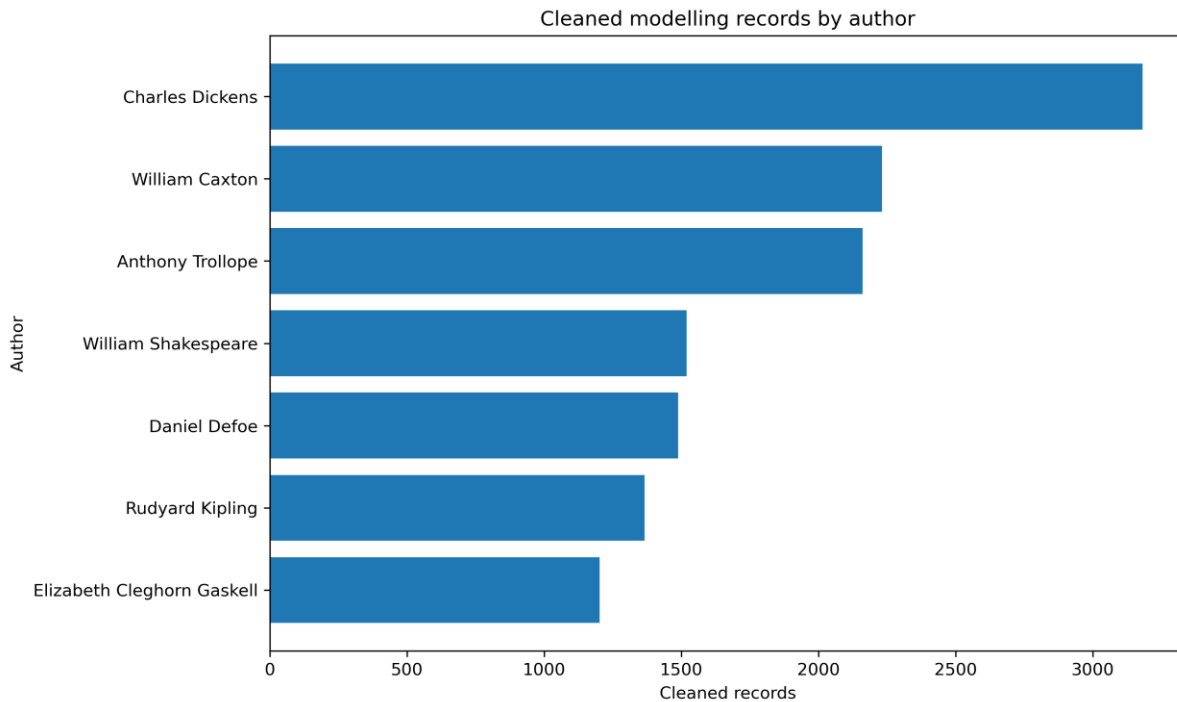


Figure 6 Final cleaned modelling records for the seven selected authors.

The cleaned class distribution remains unequal. Charles Dickens has 3,181 usable records, while Elizabeth Cleghorn Gaskell has 1,202. This imbalance directly influenced the evaluation and improvement strategy. Overall accuracy was retained as an intuitive metric, but macro F1 and balanced accuracy were treated as more important when comparing models. The improved model also used class-weighted cross-entropy so that errors on smaller training classes were not treated as less consequential simply because those classes contained fewer records. PyTorch specifically supports class weights in cross-entropy loss for imbalanced classification settings (PyTorch, 2026b).

4.3 Title diversity and leakage risk

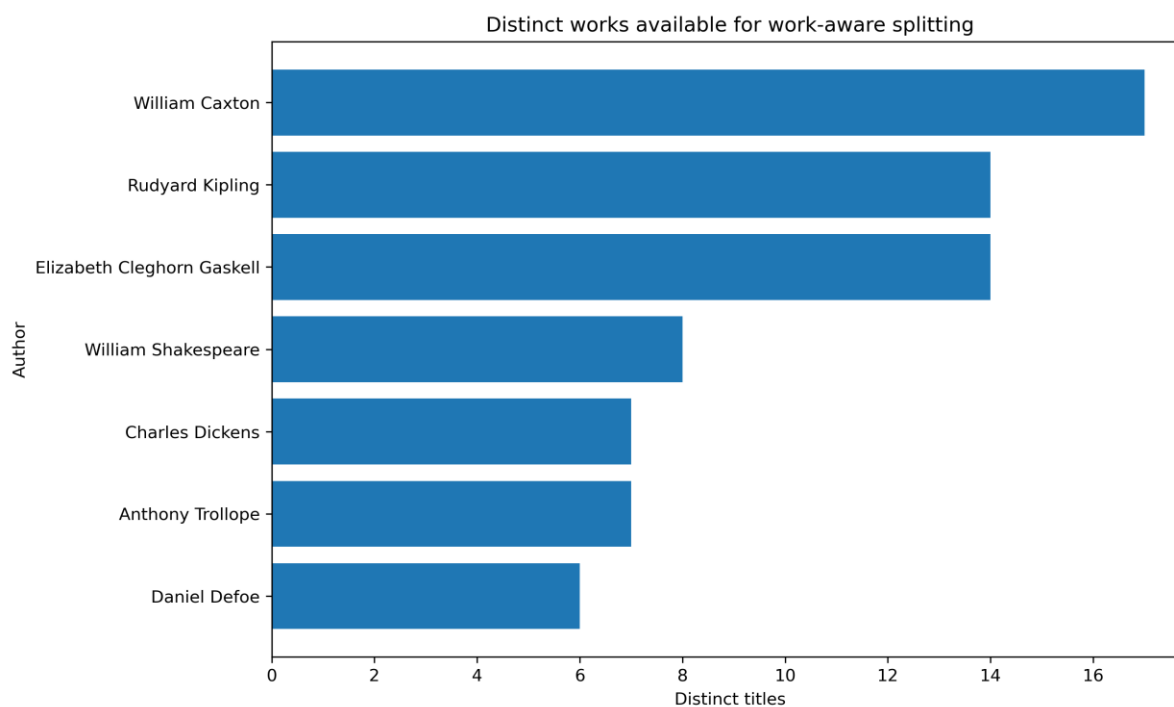


Figure 7 Distinct literary works available for each selected author.

Every selected author has at least six distinct titles. Daniel Defoe has six, Dickens and Trollope each have seven, Shakespeare has eight, Kipling and Gaskell each have fourteen, and Caxton has seventeen. This title diversity made a three-way work-aware split feasible. The aim was not to make the test set artificially difficult, but to reduce the chance that the model could recognise book-specific vocabulary from a title already represented in training.

4.4 Text-length analysis

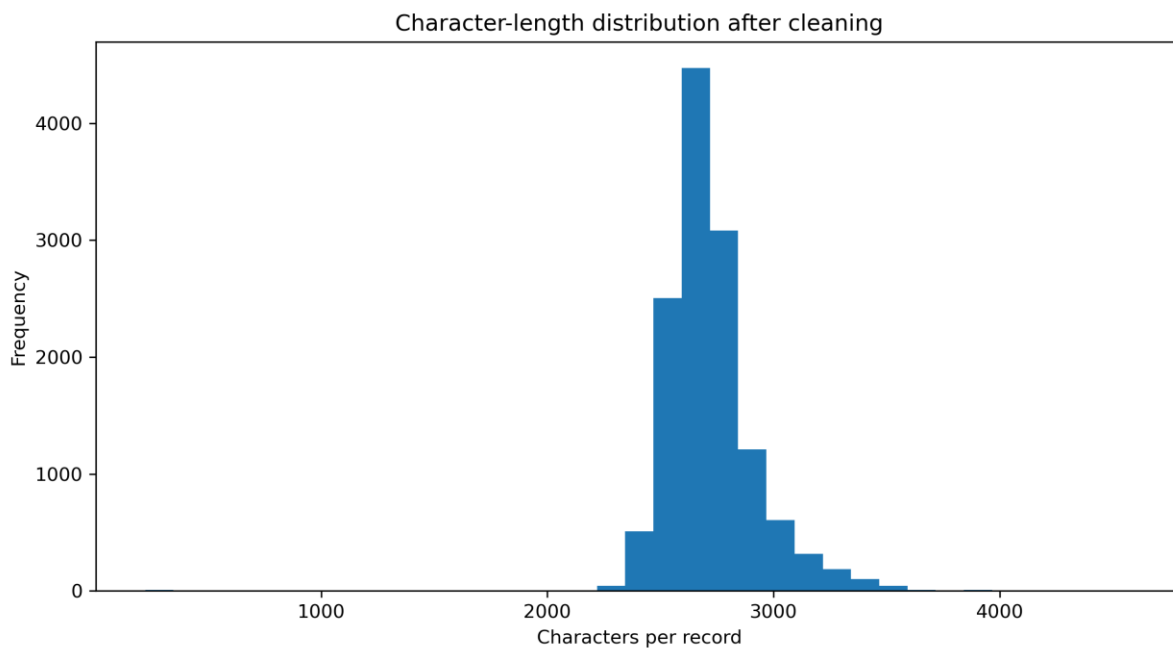


Figure 8 Character-length distribution after cleaning.

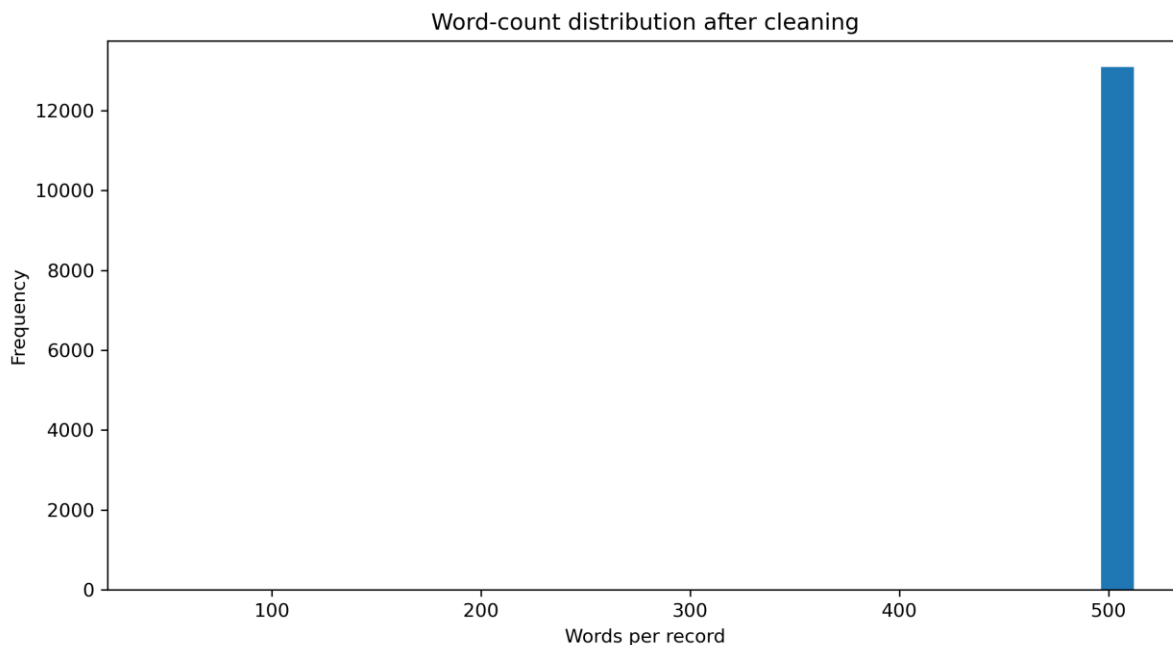


Figure 9 Word-count distribution after cleaning.

The cleaned passages have a mean character length of approximately 2,719 characters. Character length varies more than word count, but the word-count analysis reveals that TextAge is mostly segmented into fixed 512-word chunks. The median, 25th percentile and 75th percentile are all 512 words, and the maximum is also 512. The minimum is 45 words because a small number of shorter but still usable passages remain.

This finding changed the preprocessing plan. Feeding all 512 words through an LSTM would increase recurrent computation substantially on the available CPU. A fixed sequence length of 160 tokens was therefore selected as a practical compromise. The decision preserves a substantial opening portion of every passage while keeping the model computationally manageable, but it also means that later parts of longer chunks are not used and must be acknowledged as a limitation.

4.5 EDA conclusion

The EDA therefore produced four modelling decisions rather than functioning as a separate descriptive exercise. First, the author subset was restricted to frequent named authors. Second, exact duplicates and very short fragments were removed. Third, class imbalance motivated macro metrics and class weighting. Fourth, title diversity motivated a work-aware split, while the fixed 512-word source chunks motivated a shorter model sequence length. This link between observed data properties and model design is central to the quality of the final analysis.

5. Feature selection and data preparation

5.1 Feature and target selection

The predictive feature is the text passage and the target is the author. Title, category, date, decade and pre-existing text length are excluded from the neural-network input. Using title or date could allow direct or indirect shortcuts, which would weaken the claim that the model is learning from the writing itself.

Traditional p-value based forward or backward elimination is not suitable for raw token sequences. Feature selection therefore occurs through representation design: non-text metadata are withheld, the vocabulary is limited to the 30,000 most frequent training tokens, and each passage is represented as a fixed sequence of token IDs.

5.2 Tokenisation and vocabulary

The tokenizer lowercases the text and separates alphabetic words, contractions, numbers and punctuation. No stemming, lemmatisation or stopword removal is applied. Historical spelling and punctuation can carry useful stylistic or period-related information, so aggressive text normalisation would remove potentially informative patterns. Letter case, however, is intentionally normalised by lowercasing.

The vocabulary is constructed from the training partition only. Tokens that are not in the training vocabulary receive an unknown-token identifier, while shorter sequences are padded. This prevents information from validation or test passages from influencing vocabulary construction.

5.3 Work-aware split and leakage checks

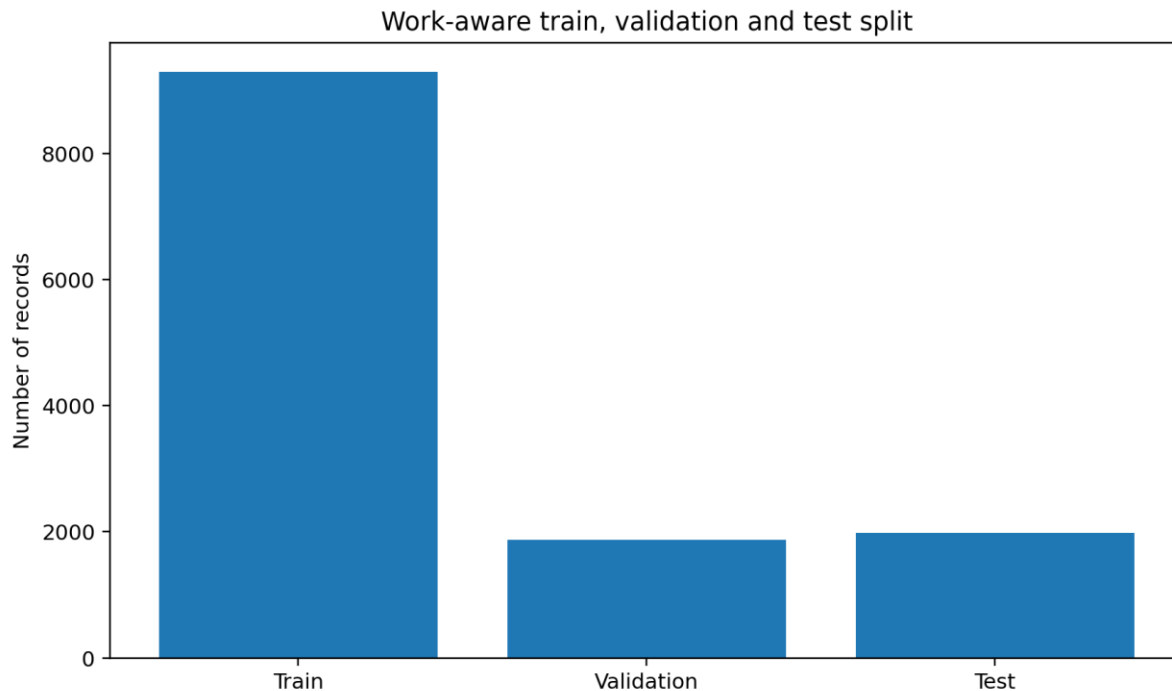


Figure 10 Record counts in the work-aware training, validation and test partitions.

```
Notebook output: leakage-aware split and preprocessing
Work-aware split
Train:      9,294  (70.68%)
Validation: 1,869  (14.21%)
Test:       1,986  (15.10%)

Leakage checks
Train-validation overlap: 0
Train-test overlap:      0
Validation-test overlap: 0
All leakage checks passed.

Vocabulary size: 30,000
Sequence length: 160
Training tensor shape: (9294, 160)
Validation tensor shape: (1869, 160)
Test tensor shape:      (1986, 160)
```

Figure 11 Notebook evidence showing the split sizes, zero author-title overlap, vocabulary size, sequence

The final partition contains 9,294 training records, 1,869 validation records and 1,986 test records, corresponding to 70.68%, 14.21% and 15.10% of the cleaned data. All seven authors are represented in every partition. Most importantly, the notebook verifies that the intersections between train-validation, train-test and validation-test author-title groups are all zero.

After tokenisation, the training tensor has shape 9,294 by 160, the validation tensor 1,869 by 160, and the test tensor 1,986 by 160. The 30,000-token vocabulary and 160-token fixed input length are then shared by both candidate neural networks so that model comparison is not confounded by different preprocessing.

6. Model training and improvement

6.1 Baseline LSTM

The baseline was intentionally simple. A trainable embedding maps token IDs to 64-dimensional vectors. A single unidirectional LSTM with 32 hidden units processes the sequence, followed by a 64-unit dense layer with ReLU activation, 30% dropout and a seven-unit output layer. Cross-entropy loss is appropriate because every passage belongs to one of seven classes, and Adam is used to update parameters during gradient-based training (PyTorch, 2026b; PyTorch, 2026c).

Table 3 Model configuration comparisons

Configuration	Baseline LSTM	Improved bidirectional LSTM
Embedding dimension	64	96
LSTM hidden units	32	32 per direction
Direction	Unidirectional	Bidirectional
Dense layer	64 ReLU units	64 ReLU units
Dropout	0.30	0.50
Loss	Cross-entropy	Class-weighted cross-entropy
Learning rate	0.001	0.0005
Maximum epochs	8	12
Training batch size	256	256
Selection criterion	Validation macro F1	Validation macro F1

Notebook output: baseline LSTM architecture and validation result

```
Baseline LSTM
Embedding(30000, 64, padding_idx=0)
LSTM(input=64, hidden=32, batch_first=True)
Dense 32 -> 64, ReLU, Dropout(0.30), Output 64 -> 7
```

Best validation result (epoch 7)

```
Validation accuracy:    0.4815
Macro precision:       0.3337
Macro recall:          0.4296
Macro F1:              0.3704
Balanced accuracy:     0.4296
```

Epoch 8 validation loss worsened after the epoch 7 minimum,
so the training routine restored the best validation-loss state.

Figure 12 Notebook evidence for the baseline LSTM architecture and best validation result.

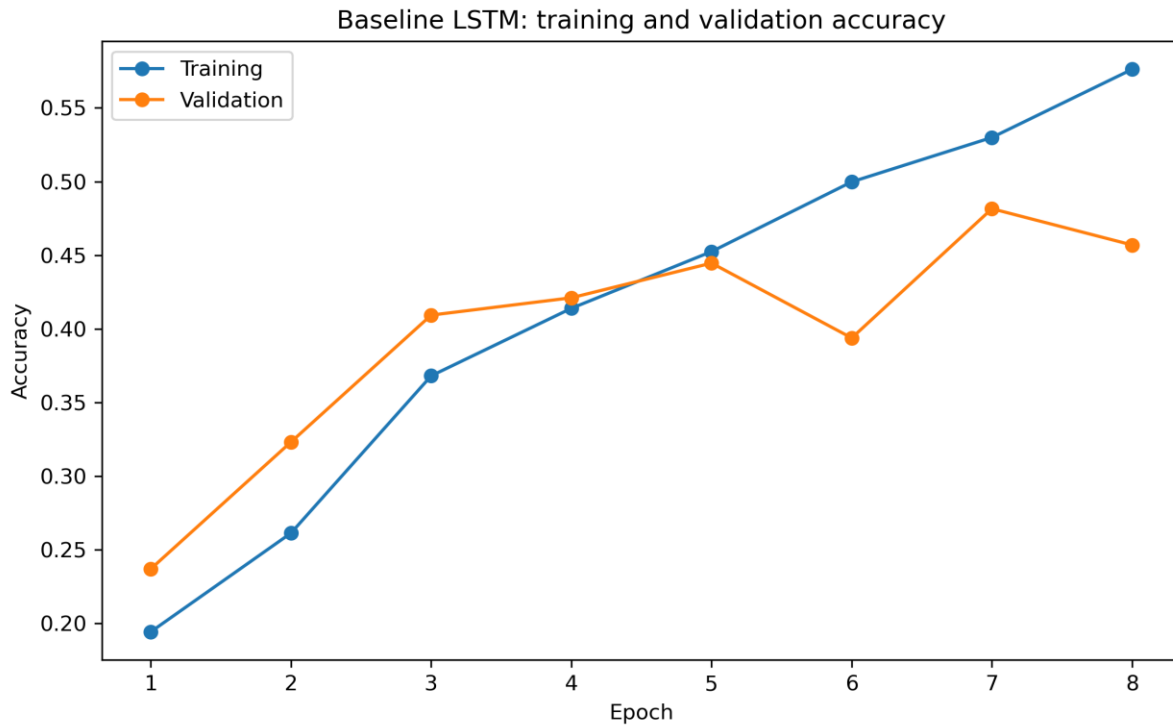


Figure 13 Baseline LSTM training and validation accuracy across epochs.

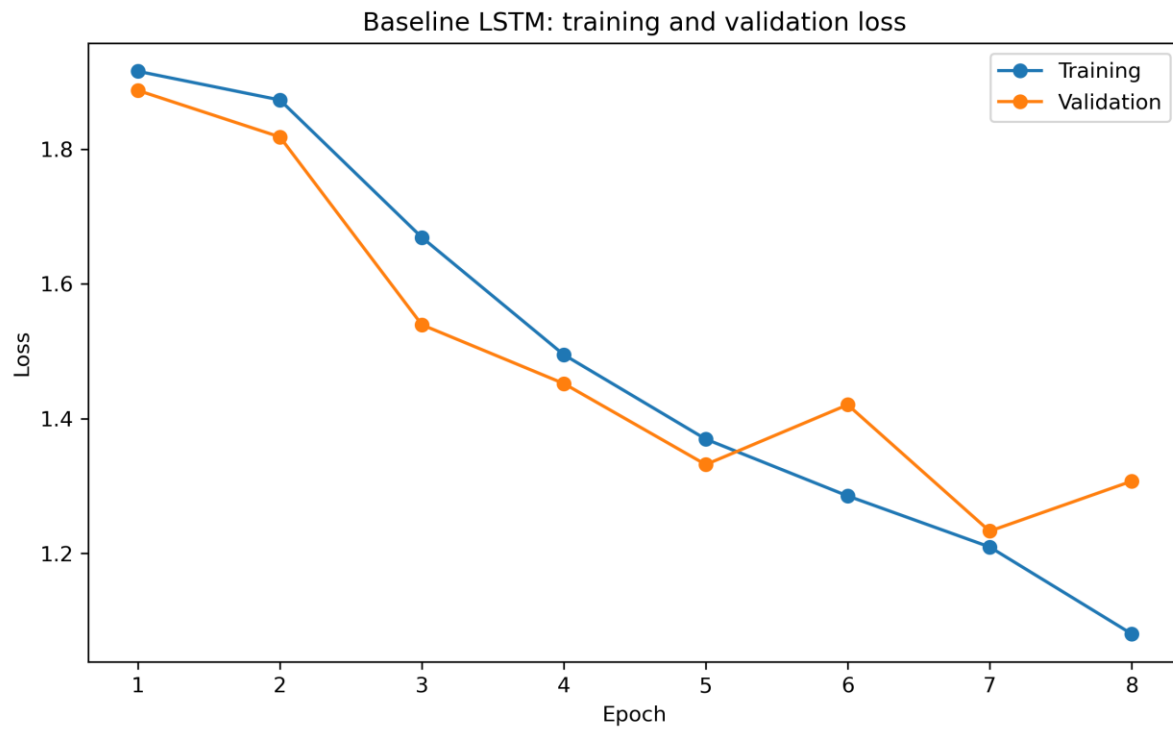


Figure 14 Baseline LSTM training and validation loss across epochs.

The baseline improves steadily during the early epochs. The best validation loss occurs at epoch 7, where validation accuracy is 48.15% and macro F1 is 0.3704. At epoch 8 the training accuracy continues to improve to 57.62%, but validation accuracy falls to 45.69% and validation loss rises. This divergence is a mild sign that additional fitting is beginning to favour the training data. The training routine restores the state associated with the lowest validation loss rather than automatically keeping the final epoch.

6.2 Improved bidirectional LSTM

The improved model was designed to address two weaknesses visible in the baseline and EDA. First, author frequencies were unequal, so the loss function received balanced class weights calculated from the training labels only. Second, a bidirectional LSTM was tested because the complete 160-token passage is available at prediction time, allowing the model to construct contextual representations from both sequence directions.

The embedding dimension was increased from 64 to 96, dropout was increased from 0.30 to 0.50, and the learning rate was reduced from 0.001 to 0.0005. These changes increase representational capacity while applying stronger regularisation and slower parameter updates.

Notebook output: improved model and validation result

```
Improved Bidirectional LSTM
Embedding(30000, 96, padding_idx=0)
Bidirectional LSTM(input=96, hidden=32)
Dense 64 -> 64, ReLU, Dropout(0.50), Output 64 -> 7
Learning rate: 0.0005
Class-weighted CrossEntropyLoss

Validation result used for model selection
Accuracy:           0.4698
Macro precision:    0.4533
Macro recall:       0.4782
Macro F1:           0.4495
Balanced accuracy:  0.4782

Selected model: Improved Bidirectional LSTM
```

Figure 15 Notebook evidence for the improved bidirectional LSTM and its validation result.

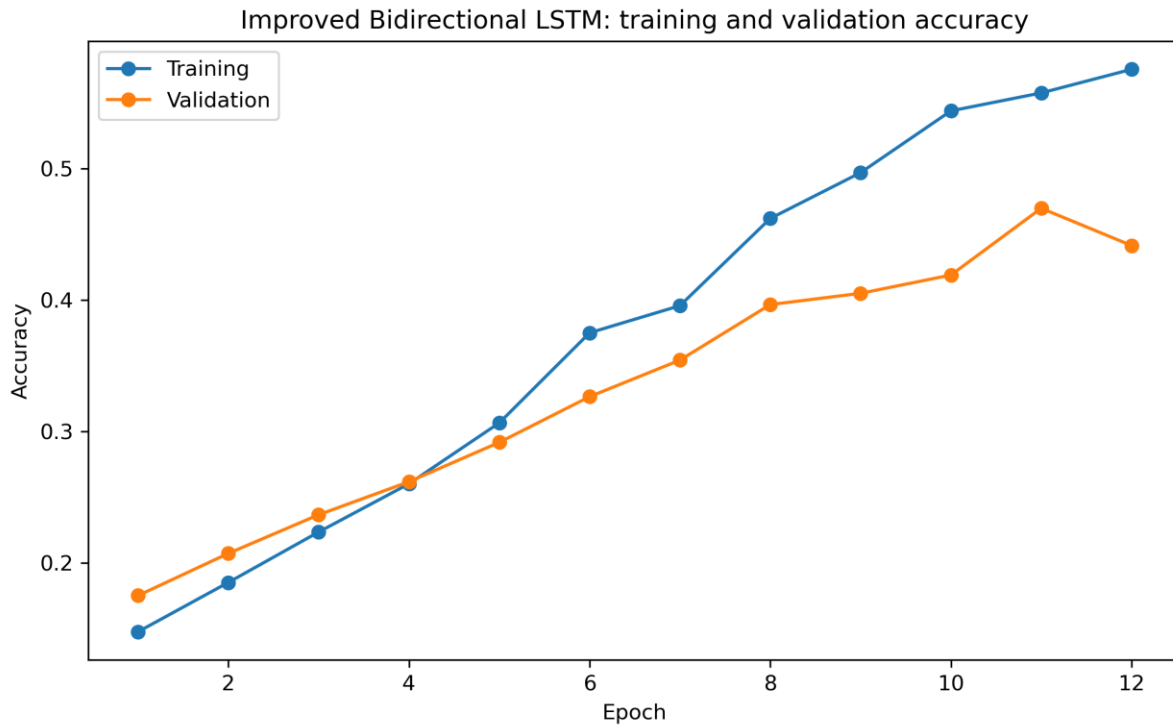


Figure 16 Improved bidirectional LSTM training and validation accuracy across epochs.

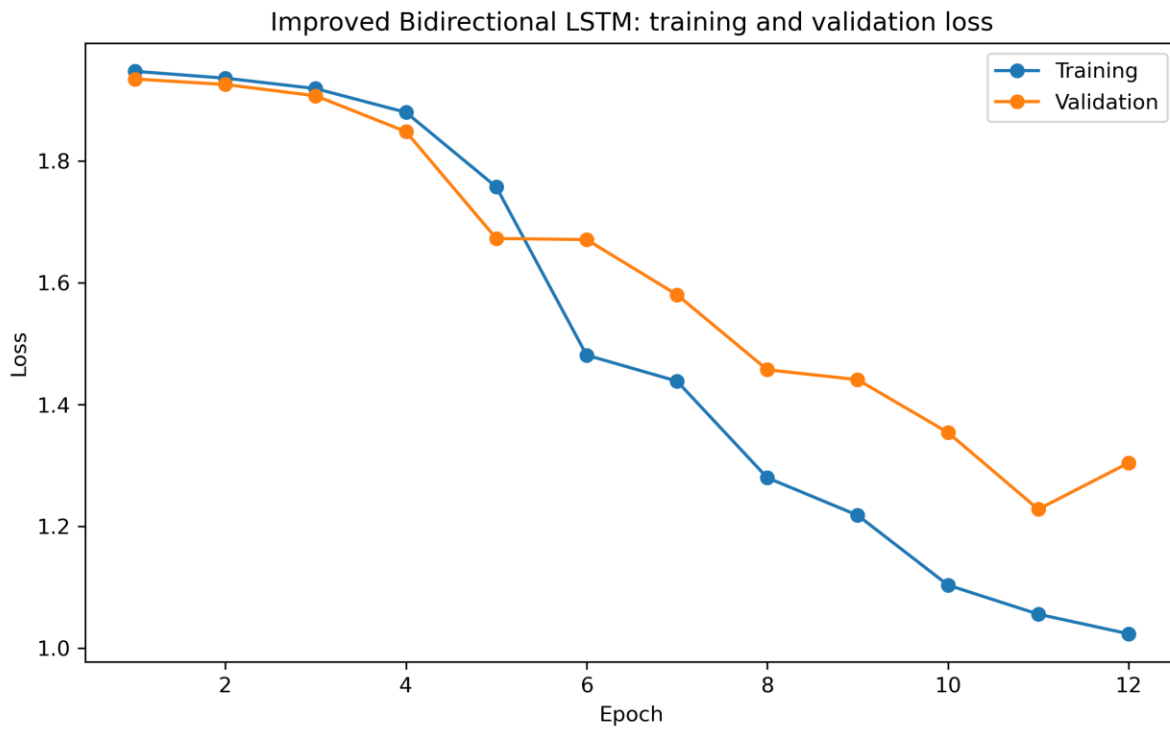


Figure 17 Improved bidirectional LSTM training and validation loss across epochs.

The improved model learns more slowly in the first few epochs, which is expected given the lower learning rate and class-weighted objective. Its best validation loss occurs at epoch 11. At that point validation accuracy is 46.98% and validation macro F1 is 0.4495. Epoch 12 produces a higher validation loss and a lower macro F1, so the best validation-loss state is restored. The important improvement is not raw accuracy; it is the substantially better balance of precision and recall across authors.

6.3 Validation-led model selection

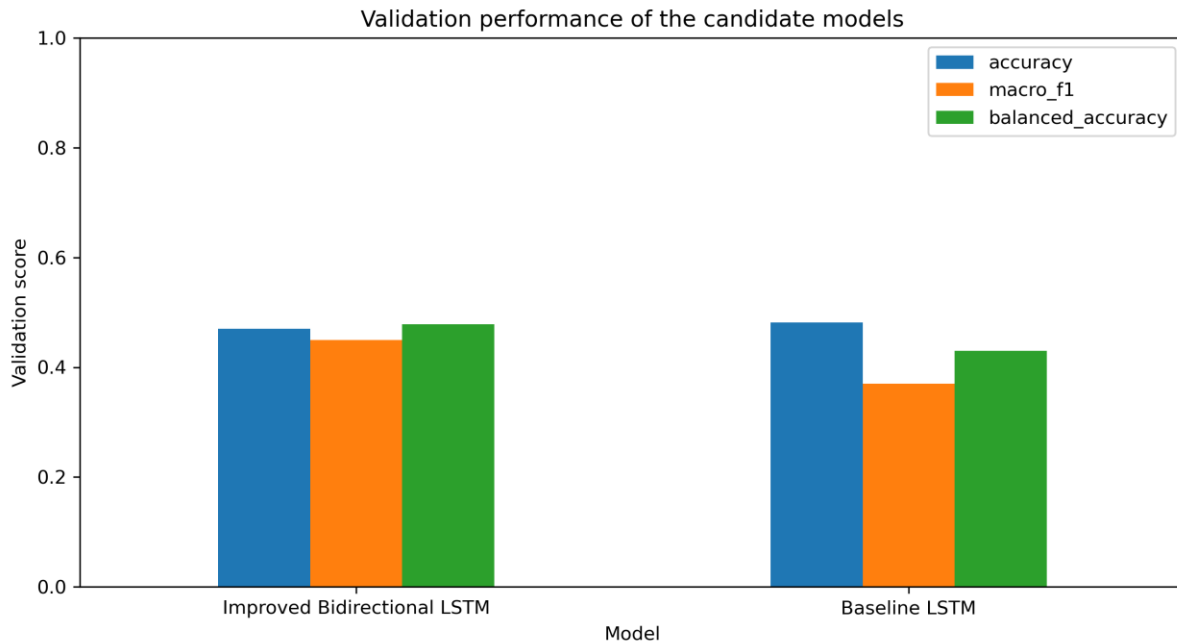


Figure 18 Validation comparison of the baseline and improved models.

The baseline validation accuracy of 48.15% is slightly higher than the improved model's 46.98%. If accuracy alone had been used, the baseline would have been selected. That would contradict the earlier plan to account for class imbalance. The improved model raises macro precision from 0.3337 to 0.4533, macro recall from 0.4296 to 0.4782 and macro F1 from 0.3704 to 0.4495. Because macro F1 was defined in advance as the main validation criterion, the improved bidirectional LSTM was selected before the test set was evaluated.

7. Final model evaluation and interpretation

7.1 Final test metrics

Notebook output: final test evaluation	
Final untouched test set	
Selected model: Improved Bidirectional LSTM	
Accuracy:	0.4391
Macro precision:	0.4428
Macro recall:	0.4779
Macro F1:	0.4429
Balanced accuracy:	0.4779
Strongest author by F1: William Caxton 0.9040	
Second strongest: William Shakespeare 0.7059	
Weakest author by F1: Daniel Defoe 0.1931	
Largest pairwise confusion:	
True author: Charles Dickens	
Predicted as: Rudyard Kipling	
Cases: 193	

Figure 19 Notebook output summarizing the final untouched test-set performance.

On the untouched test set, the selected model achieves 43.91% accuracy, 0.4428 macro precision, 0.4779 macro recall, 0.4429 macro F1 and 47.79% balanced accuracy. The closeness of macro F1 to the validation value of 0.4495 suggests that the validation result was reasonably representative of final balanced performance. The overall accuracy is lower than the baseline validation accuracy, but the test set was not used for any selection or tuning decision, so this value is the more honest estimate of final generalisation.

Table 4 Per-author performance on the untouched test set.

Author	Precision	Recall	F1 score	Support
William Caxton	0.9681	0.8478	0.9040	322
William Shakespeare	0.6036	0.8500	0.7059	120
Anthony Trollope	0.4084	0.4881	0.4447	379
Rudyard Kipling	0.2578	0.5220	0.3452	205
Charles Dickens	0.4414	0.2159	0.2899	593
Elizabeth Cleghorn Gaskell	0.1893	0.2556	0.2175	180
Daniel Defoe	0.2313	0.1658	0.1931	187

7.2 Per-author performance

William Caxton is the strongest class by a large margin, with 0.9681 precision, 0.8478 recall and 0.9040 F1. William Shakespeare is the second strongest with 0.7059 F1. These two authors also represent much older language than most of the other selected writers. Their strong separation may therefore reflect a combination of authorial style and distinctive historical vocabulary or orthography. This supports the

broader caution that high authorship-classification performance is not automatically equivalent to isolating personal style.

Anthony Trollope achieves a moderate F1 score of 0.4447, while Rudyard Kipling reaches 0.3452. Charles Dickens performs considerably worse than might be expected from his large training class: his recall is only 0.2159 and his F1 score is 0.2899. This shows why class size alone cannot explain performance. A large author class can still be difficult if unseen works use vocabulary and topics that differ from the author's training titles.

Elizabeth Cleghorn Gaskell obtains an F1 score of 0.2175 and Daniel Defoe is the weakest class at 0.1931. Defoe's recall of 0.1658 means that fewer than one in five of his held-out passages are correctly identified. These results indicate that the model is not consistently reliable across all seven writers even though a few classes are identified very well.

7.3 Confusion-matrix analysis

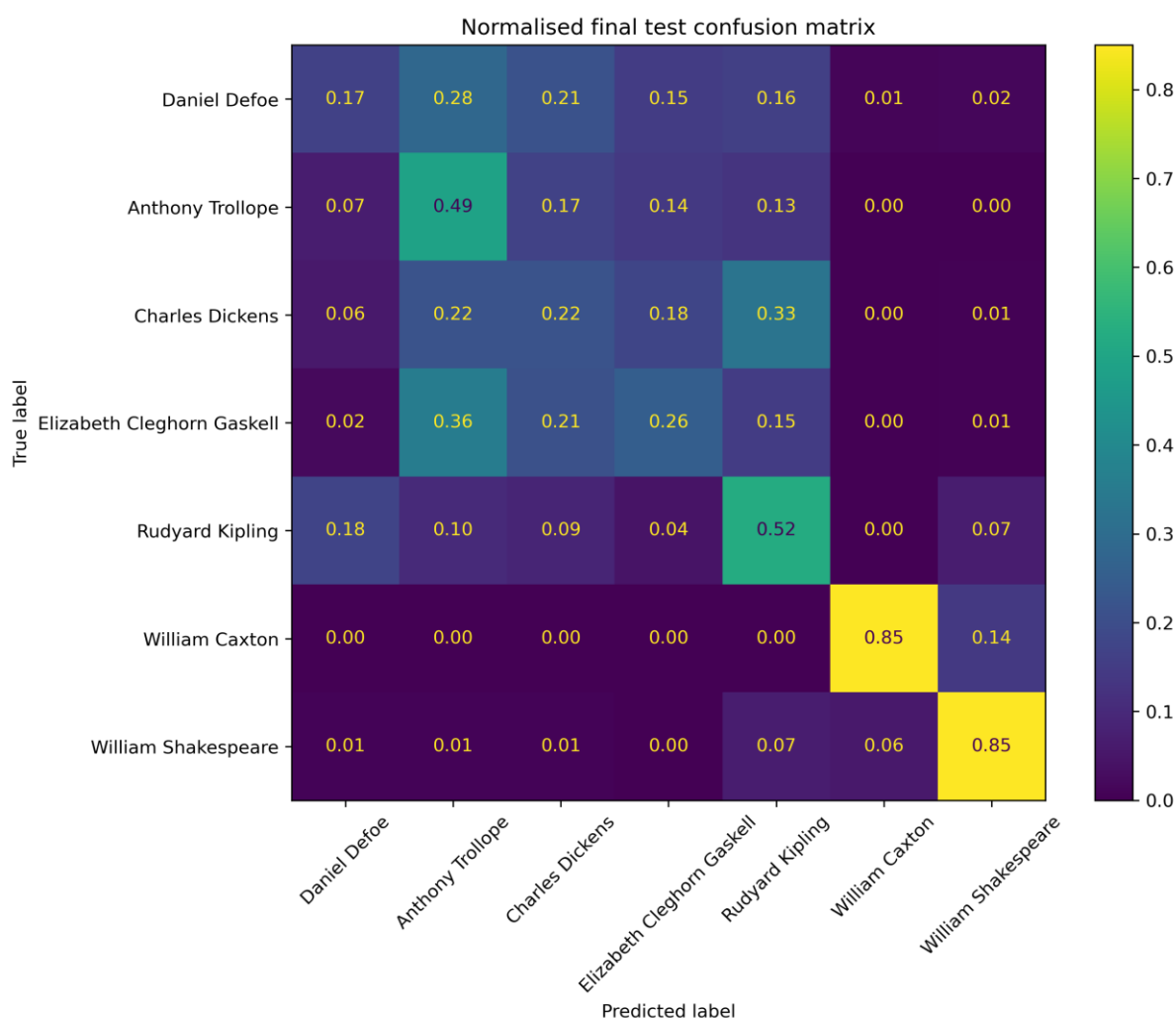


Figure 20 Normalized confusion matrix for the final test set.

The confusion matrix provides more useful diagnostic information than a single score. The largest pairwise error occurs when Charles Dickens passages are predicted as Rudyard Kipling, with 193 such cases. This contributes directly to Dickens's low recall and Kipling's relatively high recall but low precision. In other

words, the model is willing to predict Kipling often enough to recover many true Kipling passages, but it also attracts a substantial number of passages written by other authors.

The matrix also reinforces the strong separation of Caxton and Shakespeare. The contrast between these highly distinctive historical classes and the more confused nineteenth-century authors suggests that chronology and language period may be helping the classifier. A future study could control for period more tightly by selecting authors from similar decades or by evaluating style transfer across topics and works.

7.4 Final author-guessing demonstration

Notebook output: author-guessing bot demonstration

Held-out bot demonstration

Actual author: Daniel Defoe
Predicted author: William Shakespeare
Confidence: 56.09%

This example is intentionally retained even though it is incorrect.
It demonstrates the prediction function on genuinely held-out data
and provides an honest example of the model's remaining limitations.

Figure 21 Held-out demonstration of the final author-prediction function.

The retained demonstration is intentionally not replaced with a convenient correct example. The held-out passage was written by Daniel Defoe, while the model predicted William Shakespeare with 56.09% confidence. Keeping this incorrect example makes the report more credible because it demonstrates the actual function while also showing that the confidence value is not a guarantee of correctness. For a fan-facing prototype, confidence should therefore be presented as a model score rather than as certainty.

8. Discussion

8.1 Did the improvement work?

Yes, but only under the balanced criterion that was planned in advance. The improved model did not produce higher validation accuracy. Its advantage was a 0.0790 absolute increase in validation macro F1, from 0.3704 to 0.4495, together with higher macro precision, macro recall and balanced accuracy. This is exactly why relying on a single overall accuracy score would have been misleading for the unequal author distribution.

The final test macro F1 of 0.4429 is very close to the improved validation macro F1, which suggests that the model-selection decision generalised reasonably well. The test accuracy of 43.91% should still be interpreted as moderate rather than high. A random seven-class guess would have an expected accuracy of roughly 14.3% if all classes were equally likely, so the model learns meaningful signal, but it remains far from dependable across every author.

8.2 Relationship between EDA and final performance

Several final results can be traced back to the EDA. The unequal class frequencies justified class weighting and macro metrics. The title-diversity check made it possible to evaluate on unseen works rather than unseen chunks from familiar works. That stricter split likely reduced apparently easy book-specific recognition and helps explain why the final accuracy is not unusually high.

The historical range also appears important. Caxton and Shakespeare are the two strongest authors, and their language is historically distinctive relative to the later writers. This may make them easier to separate, but it also means that the model's strongest classes may be benefiting from period signals. The weak Dickens and Defoe results show that large source counts and recognisable author names do not guarantee robust held-out classification.

8.3 Limitations

- The model is closed-set and can only choose among the seven trained authors.
- Authors span very different historical periods, so chronology and orthography can act as confounding signals.
- The sequence length is capped at 160 tokens even though most source chunks contain 512 words, so later parts of each chunk are discarded.
- The title-aware split reduces work-level leakage, but author style and topic are not fully separable.
- The experiment runs on CPU with a deliberately compact LSTM. Larger representations, pretrained language models or repeated cross-validation could improve performance, but they would change the computational scope of this portfolio.
- The prediction confidence is not calibrated as a probability of authorship and should not be interpreted as forensic certainty.

8.4 Possible improvements

The next improvement should not simply be a larger network. A stronger experimental design would first control the confounding variables identified in the EDA. For example, authors could be selected from a narrower historical period, sequence length could be increased in controlled increments, and multiple work-aware splits could be evaluated to measure result stability. Character-level or subword representations may also capture spelling and punctuation patterns without relying as heavily on word vocabulary.

If more computational capacity were available, pretrained text encoders could be compared with the LSTM baseline. However, the value of such a comparison would still depend on keeping the work-aware evaluation and reporting balanced metrics rather than simply maximising one accuracy value.

9. Conclusion and client recommendation

The TextAge dataset was suitable for the Part 1 author-identification task because it supplied a large text corpus with explicit author and title labels. Spark-based inspection and cleaning produced 13,149 usable records across seven named authors, with no missing required fields, no contradictory identical texts and zero author-title overlap between the final train, validation and test partitions.

The baseline LSTM established a useful reference point, but the improved bidirectional model provided better balanced validation performance. Its validation macro F1 of 0.4495 justified selection over the baseline even though the baseline had slightly higher validation accuracy. The selected model then achieved 43.91% accuracy and 0.4429 macro F1 on the untouched test set.

For the publisher, the model is suitable as a prototype fan interaction where occasional incorrect guesses are acceptable and can even form part of the entertainment value. It is not accurate or balanced enough for high-stakes attribution. William Caxton and William Shakespeare are identified strongly, but Daniel Defoe, Elizabeth Cleghorn Gaskell and Charles Dickens remain difficult. The most defensible next step would be to improve evaluation across comparable historical periods and test longer or alternative text representations before increasing deployment scope.

10. Reproducibility and code attribution

The analysis was completed in a Jupyter Notebook using Python 3.13.9. Spark 4.2.0 was used for loading, cleaning and exploratory analysis, while PyTorch 2.14.0+cpu was used for neural-network training. The final notebook was executed from top to bottom and produced the figures and CSV outputs incorporated into this report. Apache Spark supports local Python execution through PySpark, while the Spark 4.2 documentation specifies support for modern Python versions and Java-based local execution (Apache Software Foundation, 2026).

References

- Apache Software Foundation, 2026. Installation: PySpark 4.2.0 documentation. Available at: https://spark.apache.org/docs/latest/api/python/getting_started/install.html [Accessed 5 September 2026].
- Hiray, R. et al., 2025. TextAge: A Historical Text Corpus Dataset. Kaggle. Available at: <https://www.kaggle.com/datasets/rushikeshhiray/textage-dataset> [Accessed 5 September 2026].
- Hochreiter, S. and Schmidhuber, J., 1997. Long short-term memory. Neural Computation, 9(8), pp.1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Koppel, M., Schler, J. and Argamon, S., 2009. Computational methods in authorship attribution. Journal of the American Society for Information Science and Technology, 60(1), pp.9-26. <https://doi.org/10.1002/asi.20961>.
- Paszke, A. et al., 2019. PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32, pp.8024-8035.
- Pedregosa, F. et al., 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12(85), pp.2825-2830.
- PyTorch, 2026a. LSTM: PyTorch documentation. Available at: <https://docs.pytorch.org/docs/stable/generated/torch.nn.modules.rnn.LSTM.html> [Accessed 5 September 2026].
- PyTorch, 2026b. CrossEntropyLoss: PyTorch documentation. Available at: <https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html> [Accessed 5 September 2026].
- PyTorch, 2026c. Adam: PyTorch documentation. Available at: <https://docs.pytorch.org/docs/main/generated/torch.optim.Adam.html> [Accessed 5 September 2026].
- Stamatatos, E., 2009. A survey of modern authorship attribution methods. Journal of the American Society for Information Science and Technology, 60(3), pp.538-556. <https://doi.org/10.1002/asi.21001>.
- Zaharia, M. et al., 2016. Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), pp.56-65.